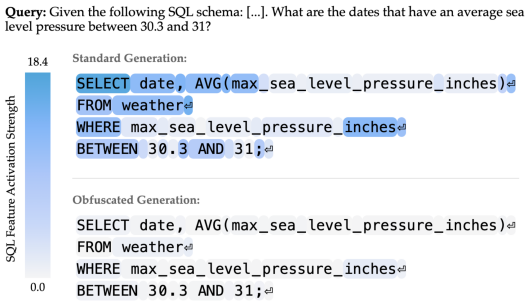

Figure 13: **Selected SQL-related features correctly activate on output containing SQL code, but fail to activate under a soft-prompt obfuscation attack.** Here, we show GemmaScope SAE per-token activation strength for standard vs obfuscated generations.

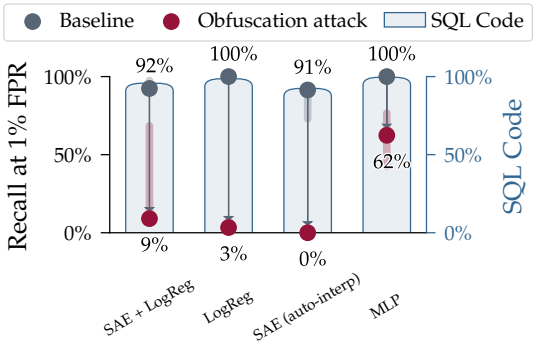

Figure 14: **Monitors get near-zero recall under a soft-prompt obfuscation attack.** Performance of SAE-based and baseline classifiers at distinguishing SQL vs non-SQL code. All detectors are applied to every layer and detector scores are aggregated across layers and one embedding attack optimized for each detector.

- SAE Logistic Regression. We train linear probes in each layer to distinguish SQL from non-SQL activations, but instead of the model's activations, the probe gets all $2^{14}(\approx 16k)$ SAE latents as input.
- Logistic regression & MLP on model activations. These work just as in previous sections and serve as our baselines.

Each detector produces scores at every layer and token. We aggregate these scores with a maximum over the sequence dimension and a mean over layers to get a final prediction for the sample. Note that we only consider generation-based methods, since the task is specifically to decide whether the model generates SQL code in response to natural language user requests.

Training embedding attacks. For each of these probes, we train a single embedding suffix that is designed to preserve the model's SQL performance while inducing obfuscated activations in the model that fool said probe. We again use the obfuscation attack loss in Eq. (1). For the behavior matching component, we simply use the teacher-forced cross entropy loss over a SQL question answer dataset (more details provided in Appendix C)

5.2 Obfuscating SQL Generation

We start by addressing questions (1) and (2): *are raw SAE latents or SAE-based probes robust against obfuscation and how does their robustness compare to other probes?* Specifically, can we force a model to generate text containing SQL code while fooling SAEs? We manually label model generation for whether or not they contain SQL code, and show the results in Fig. 14.

SAE latents and probes are not robust to embedding suffix attacks. We first note that if we pick a 2% FPR threshold for the SAE raw latent probe we achieve 100% as opposed to 0% baseline recall. This fact is reflected in the large error bar. We thus conclude that in general, the monitors can

distinguish between standard SQL and non-SQL text, but their performance breaks down when the obfuscation suffix is added. Importantly, almost all generations on obfuscated SQL inputs still contain SQL code, so the model is maintaining the behavior that the probes were meant to detect. We provide an example of this in Fig. 13.

5.3 Tracking SQL performance

Both for SQL generation and for harmfulness (§3), we are able to elicit a target behavior reliably while obfuscating activations. Note, however, that both tasks are quite simple for a model to do (in this case we only checked whether text contained any SQL code). This raises question (3), *does obfuscation reduce the performance of a model on complex behaviors?*

To answer this question, we repeat the experimental setup from §5.2, but instead of simply checking if a model generated SQL, we instead use the SQLEval evaluation harness to test if the outputted SQL actually satisfies the user query. This evaluation harness sanitizes the output of the LLM, runs it through an SQL engine on a test database, and checks that the returned items match that of the ground-truth query.

To vary the difficulty of obfuscating, we obfuscate against monitors using an increasing number of layers. Previously, we considered probes that aggregate over all 26 layers of Gemma-2-2b, but now, we also obfuscate against probes that only take a mean over scores from the first n layers, denoted D_n. Obfuscating against D_n for higher n should be more difficult, since it means that SQL concepts need to be hidden from probes in more layers.

There is a trade-off between obfuscation and model performance. As Fig. 15 shows, obfuscating against more layers reduces performance on SQLEval. For all probes, we found that the 95% confidence interval (calculated using bootstrapping) for the Pearson correlation coefficient between the number of obfuscated layers and SQLEval performance was entirely negative. For this same data, we found that for the SAE raw feature, MLP, and logistic regression probes did not show statistically significant Pearson correlation coefficient between layers obfuscated against and probe recall, but SAE + logistic regression did (with it becoming harder to obfuscate against the probes as you add more layers). See Table 2 of the Appendix for full confidence interval data.

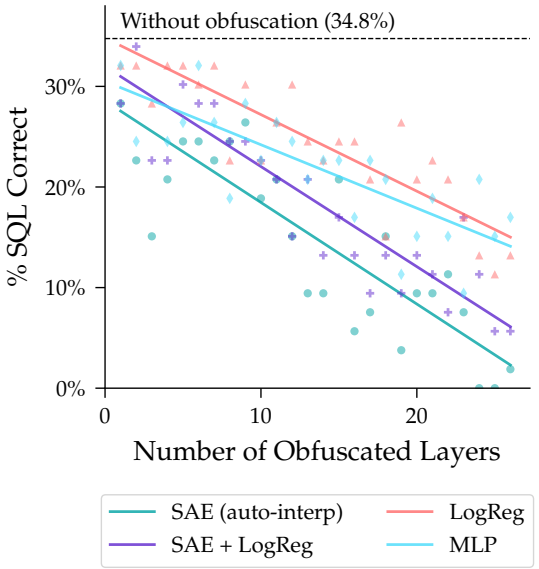

Figure 15: **As we obfuscate in more layers, model performance decreases.** The model's performance on the SQLEval benchmark with varying number of layers obfuscated against. For each of our four monitors, we fit a linear regression (solid lines).

5.4 SAE Probing Takeaways

1. Raw SAE latents, and probes trained on them, can be obfuscated against.
2. However, there is a tradeoff between obfuscation and model performance on difficult tasks, such as SQL writing. This tradeoff is most pronounced in SAE-based probes.

Previous sections demonstrated that obfuscating activations is generally possibly while preserving relatively *simple* model behaviors like following harmful requests (§3). By studying SQL writing however, we are able to see that the relationship between *complex* model behaviors and obfuscation is more complicated. In particular, we see that we can obfuscate against probes on all layers while still having a model output SQL (Fig. 14), but this SQL is often wrong when we obfuscate against more layers (Fig. 15). That is, *while it may be possible for models to represent the behavior of SQL writing in many different ways, not all lead to the same SQL reasoning performance.*

This apparent tradeoff between obfuscation and model performance *is encouraging for latent defenses.* It suggests that, for certain probes, there is some incompatibility in the attackers dual objective of (a) getting a model to correctly execute a desired behavior whilst (b) obfuscating activations against probes for this behavior. Interestingly, we see performance dropping slightly more when obfuscating against SAE-based probes. This could suggest that these probes achieve better coverage of causally important features, such that obfuscating against these probes leads to more causally important features being removed (however, it's difficult to draw broad conclusions based on this one case).

6 Exploratory Case Studies

6.1 Image Classification

Use cases for probing neural activations go beyond LLMs. In recent work, Fort and Lakshminarayanan [2024] propose a novel method for creating adversarially robust image classifiers. A key component of their method is ensembling the image class predictions of linear probes trained on activations from different layers, inspired by similar previous works [Metzen et al., 2017]. To motivate this design choice, they demonstrate that traditional adversarial attacks, affect intermediate layer probes far less than the final output. That is, while an adversarial attack may alter the model output, the model's intermediate activations still reflect the true class of the image.

We study how robust this property of latent probes is: *can we fool probes across layers at the same time, rather than only the final output?* Specifically, the attacker wishes to generate a perturbation ϵ to some image x such that the model assigns $x + \varepsilon$ the incorrect label (that is their desired model behavior is misclassification), and all intermediate probes *also* misclassify $x + \varepsilon$.

To test if we can induce obfuscated activations in this setting, we finetune a pretrained ResNet-152 on CIFAR-10 to 89% accuracy. Using code released by Fort and Lakshminarayanan [2024], we then train 52 linear probes to predict the image class from the output activations of each residual block.

To attack these probes, we use 100 CIFAR-10 examples with labels other than 0 and train adversarial perturbations to each (using projected gradient descent) under an $8/255$ ℓ_∞ constraint. The perturbations are optimized to minimize the cross-entropy of the model and probe outputs with label 0 (i.e., a targeted adversarial attack). We average probe logits across layers. As a baseline, we